Computer Organization and Architecture (CS2007)

Course Project

Team Details:

Akhil IVS – CS24B2005
B Rakesh – CS24B2039
K Siva Phaneendra – CS24B2048
L Guna – CS24B2043

---

1 Introduction

Instruction fetching remains a critical bottleneck in microprocessor architecture. As CPU clock speeds continue to outpace memory access times, the penalty for waiting on memory—particularly after a control-flow change like a branch—introduces severe pipeline stalls. When a branch is taken, the pre-fetched instructions currently in the pipeline must often be flushed, and the CPU sits idle while waiting for main memory to supply the correct target. This project explores a hardware-level solution to mitigate this latency by speculatively prefetching branch targets into an isolated, high-speed buffer closer to the CPU.

2 Problem Statement

The objective is to design and implement a Verilog hardware module that autonomously detects ARM branch instructions in the instruction stream, calculates their target addresses, and prefetches those target instructions from memory into a low-latency buffer, thereby hiding the memory access latency from the processor and minimizing idle clock cycles.

3 Implementation

The system architecture is divided into four primary Verilog modules that operate in parallel with the main CPU execution:

- **ARM Branch Predecoder:** Snoops the instruction bus. When valid, it checks bits [27:25] for the branch opcode. It computes the target PC by extracting the 24-bit offset, shifting it left by 2 to word-align it, sign-extending it to 32 bits, and adding it to the pre-computed (pc_in + 8).
- **Prefetch AXI Master:** An FSM acting as a proxy memory requester. Upon receiving a valid target signal, it initiates an AXI4-Lite read transaction to fetch the target instruction, strictly adhering to the valid/ready handshake protocol.

- **Sidecar FIFO:** A synchronous, 8-depth circular buffer utilizing N+1 pointer logic to differentiate between full and empty states. It safely isolates the timing domains between the prefetcher's memory responses and the CPU's consumption rate, and supports an asynchronous flush.
- **CPU Memory Arbiter:** A 9-state FSM that multiplexes the shared memory bus. It intercepts CPU read requests and checks the FIFO. On a cache hit, it serves the data immediately. On a miss, it routes the request directly to main memory, ensuring zero stall time is added to standard operations. It also intelligently grants bus access to the Prefetch AXI Master only during idle CPU cycles.

4 Results

The system was verified using a comprehensive testbench (`tb_prefetch_system.v`) executing a mock instruction sequence across 6 distinct phases with simulated memory access delays. The design successfully ignored standard sequential instructions (NOPs) and correctly triggered on branch opcodes. The prefetcher autonomously filled the FIFO with the target instruction, allowing a subsequent CPU fetch to execute with absolute zero memory latency. Furthermore, the system successfully demonstrated hardware stability during mispredictions by executing a 1-cycle FIFO flush and falling back to standard memory access without halting the CPU or locking the bus.

5 Challenges Faced

The most significant challenge was designing the memory arbiter. Multiplexing a single memory interface between two active masters (the CPU and the Prefetcher) required strict FSM state management to prevent bus collisions and prioritize CPU data. Additionally, implementing the AXI protocol handshake (`arvalid/arready` and `rvalid/rready` signals) required careful timing across modules to ensure data was neither overwritten nor lost during simultaneous read and write operations, especially preventing the FIFO's N+1 pointers from falsely triggering a full or empty state during concurrent access.

6 Deliverables

- **Source Code Repository:** `https://github.com/bunny501/arm-branch-prefetcher`
- **Code Explanation (with Snippets):** The project architecture consists of five core files:

  1. `arm_branch_predecoder.v`
  A combinational logic block that snoops the instruction bus, detects ARM branch opcodes (`3'b101`), and instantly calculates the target address. This parallel execution guarantees no latency is added to the critical path.

```
always @(*) begin
    if (inst_valid) begin
        if (inst[27:25] == 3'b101 && inst[31:28] != 4'b1111)
           begin
```

```
            is_br_c = 1'b1;
            tgt_pc  = (pc_in + 32'd8) + {{6{inst[23]}}, inst
               [23:0], 2'b00};
        end else begin
            is_br_c = 1'b0;
            tgt_pc  = 32'd0;
        end
    end
end
```

2. **`prefetch_sidecar_fifo.v`**

An 8-depth synchronous circular queue storing a 64-bit payload. It uses N+1 pointer logic (4-bit pointers for an 8-slot array) for full/empty states and features a 1-cycle flush for misprediction recovery.

```
reg [3:0] wp;
reg [3:0] rp;

assign empty = (wp == rp);
assign full  = (wp[3] != rp[3]) && (wp[2:0] == rp[2:0]);

always @(posedge clk or posedge flush) begin
    if (flush) begin
        wp <= 4'b0;
        rp <= 4'b0;
    end
end
```

3. **`prefetch_axi_master.v`**

A 3-state FSM that acts as a proxy memory requester, fetching the calculated target instruction via the AXI protocol once the bus is granted.

```
parameter IDLE = 2'b00, REQ = 2'b01, WAIT = 2'b10;

always @(posedge clk) begin
    case (state)
        IDLE: if (valid_target) begin
            mem_araddr <= target_pc;
            mem_arvalid <= 1'b1;
            state <= REQ;
        end
        REQ: if (mem_arready) begin
            mem_arvalid <= 1'b0;
            state <= WAIT;
        end
        WAIT: if (mem_rvalid && mem_rresp == 2'b00) begin
            fifo_push_data <= mem_rdata;
            state <= IDLE;
        end
    endcase
end
```

4. `cpu_memory_arbiter.v`
A 9-state FSM multiplexing the main memory bus. It prioritizes CPU requests, bypasses memory entirely on a FIFO hit, and seamlessly routes misses back to main memory.

```
always @(posedge clk) begin
    if (cpu_req_valid) begin
        if (!fifo_empty && (cpu_req_addr == fifo_dout[63:32]))
           begin
            cpu_rdata  <= fifo_dout[31:0];
            fifo_pop   <= 1'b1;
            cpu_rvalid <= 1'b1;
        end else begin
            mem_araddr  <= cpu_req_addr;
            mem_arvalid <= 1'b1;
        end
    end
end
```

5. `tb_prefetch_system.v`
The top-level testbench featuring a custom 64-word mock memory with artificial cycle delays, used to validate the system across 6 distinct test phases.

- **Experimental Results:**

```
PS C:\Users\akhil\Downloads\COA\ARM Branch Prefetcher> wsl
akhil@AkhilAsus:/mnt/c/Users/akhil/Downloads/COA/ARM Branch Prefetcher$
v prefetch_axi_master.v prefetch_sidecar_fifo.v
akhil@AkhilAsus:/mnt/c/Users/akhil/Downloads/COA/ARM Branch Prefetcher$

=== Branch-Based Prefetcher TB ===

[Phase 0] Reset
VCD info: dumpfile tb_prefetch_system.vcd opened for output.

[Phase 1] NOP at 0x00
  CPU fetch 0x00000000 → 0xe3a00000
  PASS: got NOP instruction
  PASS: predecoder silent on NOP
  PASS: FIFO still empty

[Phase 2] Branch at 0x04
  CPU fetch 0x00000004 → 0xea000002
  PASS: got branch instruction
  PASS: predecoder flagged branch (sticky)
  PASS: target computed = 0x14 (sticky)

[Phase 3] Waiting for prefetch fill...
  PASS: FIFO filled by prefetcher
  PASS: FIFO head PC = 0x14
  PASS: FIFO head instr = MOV R1,#1

[Phase 4] CPU fetch 0x14 (expect FIFO hit, no memory access)
  CPU fetch 0x00000014 → 0xe3a01001
  PASS: got MOV R1,#1 from FIFO
  PASS: mem_arvalid never went high — memory bypassed!
  PASS: FIFO empty after pop

[Phase 5] Flush on misprediction
  CPU fetch 0x00000004 → 0xea000002
  PASS: FIFO re-filled before flush
  PASS: FIFO empty 1 cycle after flush

[Phase 6] Miss at 0x08 (not in FIFO)
  CPU fetch 0x00000008 → 0xe3a00001
  PASS: got MOV R0,#1 from memory
  PASS: mem_arvalid went high — miss path works

=== RESULTS: 16 passed, 0 failed ===
ALL TESTS PASSED
tb_prefetch_system.v:275: $finish called at 736000 (1ps)
akhil@AkhilAsus:/mnt/c/Users/akhil/Downloads/COA/ARM Branch Prefetcher$
```

- **Zero-Latency Verification:** During phase 4 of the testbench, waveform analysis confirmed that upon a cache hit, the `cpu_rvalid` signal asserted immediately on the next clock edge. The arbiter successfully intercepted the request and bypassed memory entirely, proven by the fact that the `mem_arvalid` signal never triggered on the main memory bus. This saved the standard multi-cycle memory access delay.
- **Fault Tolerance Validation:** Phase 5 execution proved that triggering the asynchronous `flush` input successfully cleared the FIFO pointers (`rp` and `wp` fell to `4'b0000`) in a single clock cycle without stalling the main memory bus or locking up the FSM, proving graceful degradation on branch mispredictions.

- **Inferences:**
  - The hardware successfully hides memory latency for correctly predicted branches, effectively converting expensive multi-cycle memory fetch penalties into zero-cycle fetches from the CPU's perspective. This decoupling of memory latency from the CPU's execution pipeline is crucial for scaling processor frequencies.
  - The N+1 pointer logic implementation is robust against simultaneous read-/write operations, preventing data corruption during high-throughput background fetching when the CPU and Prefetcher act at the same time.
  - The strict CPU-priority logic designed within the arbiter prevents bus deadlocks, ensuring the CPU is never starved by the prefetcher's autonomous background requests.
- **Replication Commands:** To compile and simulate this project in a standard Linux/WSL environment using Icarus Verilog, execute the following commands in the terminal:

```
iverilog -o prefetch_sim.vvp tb_prefetch_system.v
   arm_branch_predecoder.v cpu_memory_arbiter.v prefetch_axi_master.
   v prefetch_sidecar_fifo.v
vvp prefetch_sim.vvp
```

7 Future Work

Future iterations of this design could integrate a Branch History Table (BHT) using a 2-bit saturating counter to enable dynamic branch prediction, allowing the prefetcher to handle conditional branches more intelligently based on past execution history. Additionally, the sidecar FIFO could be widened to support superscalar fetch widths, retrieving 2 or 4 contiguous instructions per cycle to further saturate the CPU pipeline and improve Instruction Per Cycle (IPC) throughput.